

SW프로젝트1 과제

휘핑크림팀

22100182 이나영(팀장), 22101796 장소예

2025. 06.13

1. 반응형 웹 기능 구현

- 뷰포트 설정

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` //브라우저
너비를 사용자의 장치 너비에 맞게 표시하면서 초기화면 배율을 1.0으로 설정하여 모든
디바이스에서 적절한 화면 비율로 보이도록 설정

*width=device-width : 화면 너비를 장치의 독립 픽셀 너비로 설정

*initial-scale=1.0 : 확대/축소 없이 1:1 비율로 초기화면 표시

- 미디어쿼리

/* 984px ~ 1243px: 작은 데스크탑/큰 태블릿 */

```
@media screen and (min-width: 984px) and (max-width: 1243px) {  
  /* 폰트 크기 축소, 여백 조정 */  
  .header_img { width: 100px; height: 100px; }  
  nav ul li a { font-size: 15px; }  
}
```

/* 821px ~ 983px: 중형 태블릿 */

```
@media screen and (min-width: 821px) and (max-width: 983px) {  
  /* 더 작은 폰트, 메뉴 간격 축소 */  
  .header_img { width: 90px; height: 90px; }  
  nav ul li { margin-right: 5px; }  
}
```

/* 700px ~ 820px: 소형 태블릿 */

```
@media screen and (min-width: 700px) and (max-width: 820px) {  
  /* 모바일 친화적 레이아웃 시작 */  
  #header2 { top: -15px; right: 5px; }  
  nav ul { margin-top: -40px; }  
}
```

```
/* 450px ~ 699px: 대형 스마트폰 */
```

```
@media screen and (min-width: 450px) and (max-width: 699px) {
```

```
/* 모바일 전용 스타일 */
```

```
header { height: 50px; }
```

```
.header_img { width: 60px; height: 60px; }
```

```
nav ul li a { font-size: 9px; }
```

```
}
```

화면 크기에 따라 5가지 구간으로 구분하여 다른 스타일을 적용했다.

- 유동적인 레이아웃

```
display: flex;
```

```
flex-direction: column/row;
```

```
justify-content: center;
```

```
align-items: center;
```

flexbox를 활용해 배치가 유연하게 되도록 했고 %와 auto를 사용하여 창의 넓이에 따라 크기가 유동적으로 바뀌게 설정했다.

- 반응형 이미지 처리

```
img {
```

```
width: 100%;
```

```
height: auto;
```

```
max-width: 100%;
```

```
}
```

이미지가 컨테이너에 맞춰 자동적으로 크기가 조정되게 했다.

- 반응형 단위 사용

```
font-size: clamp(14px, 1.2vw, 18px);
```

```
width: clamp(160px, 22vw, 250px);
```

뷰포트 너비의 1%를 나타내는 상대적 단위인 vw (viewport width)를 사용하여 디바이스 크기에 따라 자동으로 스케일이 조정되도록 하였다.

clamp()함수란 3개의 값을 받아 최소값, 기준값, 최대값을 지정하는 함수인데 clamp(min, val, max)형식으로 사용한다.

2. 카카오페이

❖ 결제 페이지 (PayBank.jsp)

사용자가 결제 방식을 "카카오페이"로 선택하면,
카카오페이 결제 준비를 처리하는 **kakao_ready.jsp**로 이동한다.

❖ 카카오페이 결제 대기 페이지 (kakao_ready.jsp)



- 요청 처리 준비

```
<%@ page contentType="application/json;charset=UTF-8" pageEncoding="UTF-8" %>
```

//jsp페이지의 응답 형식을 json으로 지정

```
request.setCharacterEncoding("UTF-8"); //요청 바디(json)를 utf-8로 인코딩하여 읽음
```

```
JSONObject result = new JSONObject();
```

- 요청 바디(json) 읽기 및 파싱

```
StringBuilder sb = new StringBuilder();
```

```
BufferedReader reader = request.getReader();
```

```
String line;
```

```
while ((line = reader.readLine()) != null) {
```

```
    sb.append(line);
```

```
}
```

```
reader.close();
```

```
System.out.println("받은 JSON 데이터: " + sb.toString());    //문자열로 조합하여 sb에
```

```
저장하고 로그로 출력
```

```
JSONParser parser = new JSONParser();
```

```
JSONObject json = (JSONObject) parser.parse(sb.toString());    //받은 문자열을
```

```
JSONObject로 파싱
```

```
String userId = (String) session.getAttribute("sid");
```

```
...
```

- 결제 파라미터 설정

```
String itemName = "Order Cake";    //결제항목의 이름을 Order Cake로 설정
```

```
String partner_order_id = "ORD" + System.currentTimeMillis();    //고유 주문 번호를
```

```
생성(ORD + 현재 시간 밀리초)
```

```
String finalAmount = (String) json.get("ordPay");    //JSON에서 ordPay(결제 금액)을 꺼냄
```

- KakaoPay의 결제 준비 API 호출

```
URL url = new URL("https://open-api.kakaopay.com/online/v1/payment/ready");
```

```
URLConnection conn = (URLConnection) url.openConnection();
```

```
//URLConnection으로 KakaoPay의 결제 준비 API를 호출
```

```
conn.setRequestMethod("POST");
```

```
conn.setRequestProperty("Authorization", "SECRET_KEY ...");
```

```
conn.setRequestProperty("Content-Type", "application/json");
conn.setDoOutput(true);
```

```
JSONObject payRequest = new JSONObject();
payRequest.put("cid", "TC0ONETIME");
payRequest.put("partner_order_id", partner_order_id);
payRequest.put("partner_user_id", userId);
payRequest.put("item_name", itemName);
payRequest.put("quantity", 1);
payRequest.put("total_amount", Integer.parseInt(finalAmount));
payRequest.put("tax_free_amount", 0);
payRequest.put("approval_url", "http://localhost:8080/onlycake/KakaopayOK.jsp");
payRequest.put("cancel_url", ...);
payRequest.put("fail_url", ...);
```

```
OutputStream os = conn.getOutputStream();
os.write(payRequest.toString().getBytes("UTF-8"));
os.flush();
os.close();
```

결제 요청 JSON (총 결제 금액, 사용자 ID, 주문 ID 포함)을 작성하고 전송한다.

- 세션에 필요한 정보 저장

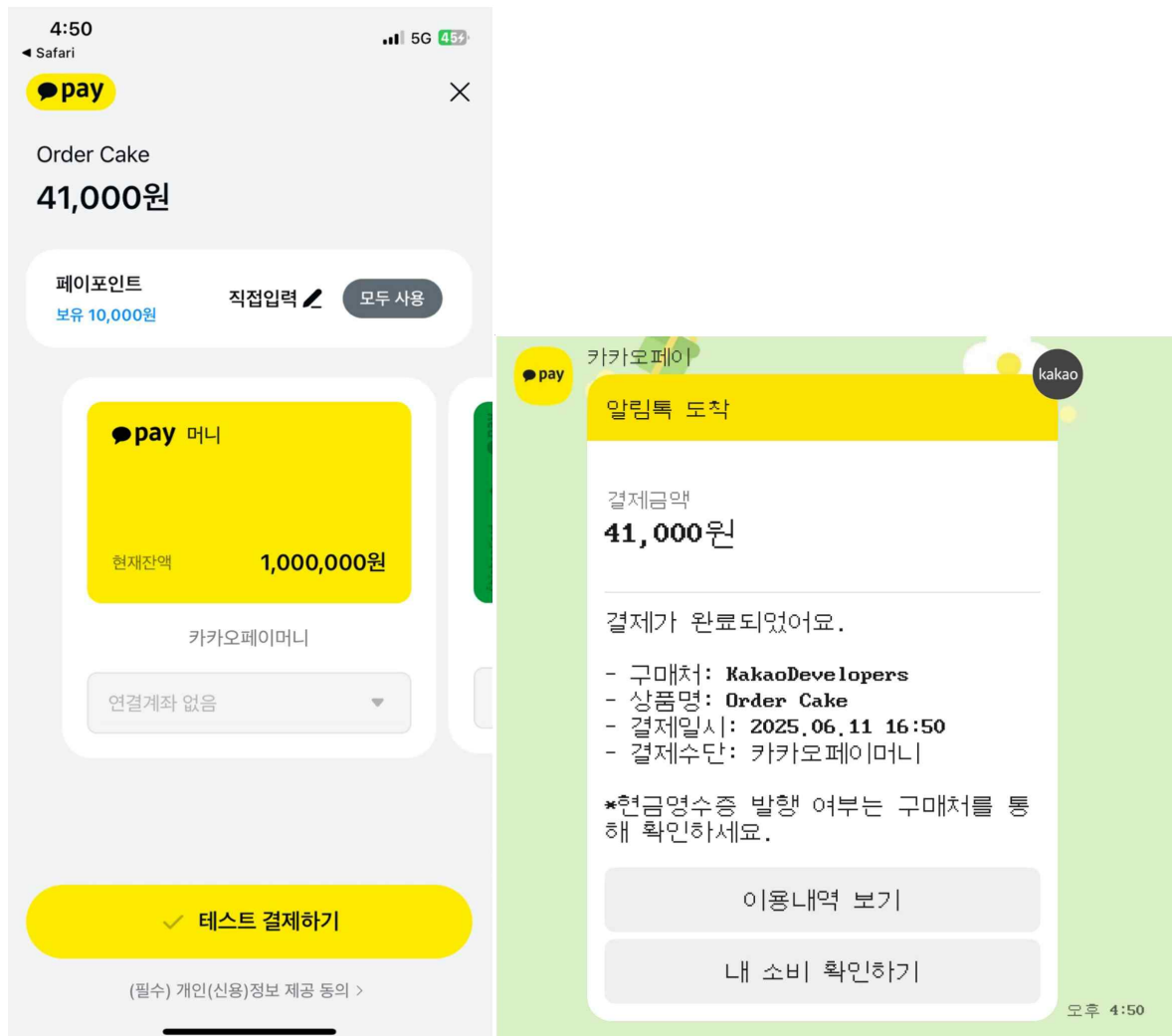
```
session.setAttribute("tid", kakaoRes.get("tid"));
session.setAttribute("order_id", partner_order_id);
session.setAttribute("kakaoPayData", json);    // 배송 및 상품 정보 저장
```

- 클라이언트에 리디렉션 URL 전달

```
result.put("redirect", kakaoRes.get("next_redirect_pc_url"));
out.print(result.toJSONString());
```

결제 승인에 필요한 tid, 주문번호 등을 세션에 저장하고 응답 JSON에 리디렉션 URL을 넣어 클라이언트에 전송한다.

❖ 카카오페이 결제 승인 처리 및 주문 저장 (KakaopayOK.jsp)



- 승인 요청 준비

```
String pgToken = request.getParameter("pg_token");
```

```
String tid = (String) session.getAttribute("tid");
```

...

먼저 초기 세션, 파라미터를 확인한다.

- 승인 요청


```
URL url = new URL("https://open-api.kakaopay.com/online/v1/payment/approve");
```

```
...
```

```
approveData.put("pg_token", pgToken);    //카카오페이 서버에 결제 승인을 요청
```

```
String ordNo = orderId;
```

```
String memName = (String) session.getAttribute("memName");
```

```
...
```

- 주문 정보 추출

```
JSONArray ctNosArray = (JSONArray) kakaoPayData.get("ctNos");
```

- 장바구니 상품 조회

```
SELECT c.*, p.prdName, p.prdPrice, ...
```

```
FROM cart c JOIN cake p ON c.prdNo = p.prdNo
```

```
WHERE c.ctNo IN (?, ?, ...)
```

- 주문 정보 저장

```
INSERT INTO orderinfo (...)
```

결제 및 배송 정보를 주문 정보를 저장하는 테이블인 orderinfo에 저장하고,

```
INSERT INTO productDetail (...)
```

상품의 상세 옵션 (크기, 맛, 과일 등)을 상품의 상세 정보를 저장하는 테이블인 productDetail에 저장한다.

- 장바구니 정리 및 상품 판매량 업데이트

```
DELETE FROM cart WHERE ctNo IN (?, ?, ...)    //주문 완료된 상품들을 장바구니에서 제거
```

```
UPDATE cake SET prdCount = prdCount + ? WHERE prdNo = ?    //해당 상품의 누적  
판매량(prdCount)을 수량만큼 증가
```

- 세션 정리 및 결과 페이지 이동

```
session.removeAttribute("tid");
```

...

```
window.location.href = "PayResult.jsp?ordNo=...";
```

결제 관련 세션 정보를 제거하고 사용자에게 결제 완료 안내 메시지를 띄운 후 결과 페이지인 PayResult.jsp로 이동시킨다.

나의 케이크

인기 케이크

6

오로이 케이크

행복한 케이크

고객 후기

🔍

🔍

🔍

주문 영수증

장바구니 ● 결제하기 ● 주문완료

주문번호ORD1749628220715

주문일자2025-06-11 16:50:55

수령인rtt

주소경기 평택시 평택로 00번길 00 동 아파트 101동 1002호

전화번호01012345678

배송 요청사항

결제 수단카카오페이

주문자명

상품 목록

디자인 케이크1개

바닐라시트 + 녹차 앙크림 / 3호(+10,000) / 체리(+5,000) / 별미치 모양 / 회오리 모양 / 하트츠(+1,000) / 보냉포장 X (5~8월에 선택시 자동 주문취소) / 2025-06-17 (발송예정일)

결제 금액41,000원

주문자명

- PayResult.jsp

3. 메인페이지에 있는 3D 공간



3D 공간은 블렌더에서 손수 제작했고 이 3D공간을 웹 환경에서 시각화하고 제어할 수 있게 [Three.js](#)를 이용하여 구현했다.

- 캔버스 및 카메라 설정

```
const canvasWidth = 1440;    //캔버스 너비를 1440px로 고정
const originalAspectRatio = 1440 / 900;    //가로 세로 비율을 16:10으로 유지
const canvasHeight = canvasWidth / originalAspectRatio * 0.7;    //세로 크기를 70%로
                                축소하여 원하는 길이로 조정
```

```
const scene = new THREE.Scene();    //3D 장면 (scene)객체 생성
const camera = new THREE.PerspectiveCamera(45, canvasWidth / canvasHeight, 0.1,
1000);    //원근 카메라 생성 PerspectiveCamera(시야각, 종횡비, 렌더링 범위)
```

```
// 초기 카메라 위치 및 target 설정
```

```
const initialCameraDistance = 0.85;
const cameraOffsetX = 0;
const targetOffsetY = -0.15;
```

```
camera.position.set(cameraOffsetX, 0.2, initialCameraDistance);    //카메라는 (x=0, y=0.2, z=0.85) 위치에 배치
const target = new THREE.Vector3(0, targetOffsetY, 0);    //카메라가 바라보는 대상(target)**은 아래쪽으로 살짝 오프셋된 (0, -0.15, 0)
camera.lookAt(target);
```

- 사용자 제어 (회전, 줌 기능)

```
const controls = new THREE.OrbitControls(camera, renderer.domElement);
//OrbitControls를 사용해 마우스로 카메라 조작 가능
controls.target = target;    //회전 중심을 아까 설정한 벡터 (0, -0.15, 0)으로 지정
```

```
controls.enableDamping = true;    // 부드러운 감속 효과
controls.dampingFactor = 0.05;
controls.rotateSpeed = 1.0;
controls.enableZoom = true;
controls.enablePan = true;
```

```
controls.minDistance = 0;    //줌 거리 제한
controls.maxDistance = 1.5;
controls.zoomSpeed = 0.7;
```

```
controls.minPolarAngle = Math.PI / 5;    // 상하 회전 제한 (36도 ~ 45도)
controls.maxPolarAngle = Math.PI / 4;
controls.minAzimuthAngle = -Math.PI / 18;    // 좌우 회전 제한
controls.maxAzimuthAngle = Math.PI / 3;
```

사용자가 시점을 과하게 벗어나지 않게 제한 범위 내에서만 줌, 회전, 패닝이 가능하게 설정했다.

- 조명

```
const pointLight = new THREE.PointLight(0xFFB6C1, 1.3, 25);    //핑크톤의 조명을 밝기 1.3, 감쇠 거리 4, 감쇠율 2로 설정
```

#FFB6C1

pointLight.position.set(0, 4, 0); //포인트 조명을 y=4에 위치(장면 위에서 중앙을 비추는 조명)

scene.add(pointLight);

const ambientLight = new THREE.AmbientLight(0xFFB6C1, 0.3); //앞과 동일한 색상의 밝기 정도가 0.3인 주변광을 생성

scene.add(ambientLight);

const hemiLight = new THREE.HemisphereLight(0xFFEACF, 0x806040, 0.5);

//상단(`0xFFEACF`)과 하단(`0x806040`) 색상을 다르게 지정하여 자연광과 지면 반사 효과 구현

#FFEACF

#806040

scene.add(hemiLight);

const softSpotLight = new THREE.SpotLight(0xFFEACF, 1.0, 20, Math.PI / 6, 0.5); //new THREE.SpotLight(색상, 밝기 강도, 밝기 거리, 조명 각도, 반 그림자(빛이 중심에서 가장자리로 갈수록 얼마나 부드럽게 약해지느냐));

softSpotLight.position.set(2, 5, 5); //약간 대각선 위에서 비춤

softSpotLight.castShadow = true;

scene.add(softSpotLight);

- 렌더링

const renderer = new THREE.WebGLRenderer({ antialias: true }); //WebGLRenderer를 생성하고 캔버스를 HTML요소 #3d에 추가하고 계단 현상을 줄이기 위해 안티앨리어싱을 켜

renderer.setSize(canvasWidth, canvasHeight); //렌더러 크기 설정

renderer.setPixelRatio(window.devicePixelRatio); //디바이스의 픽셀 밀도에 따라 렌더링 해상도 조절

document.getElementById('3d').appendChild(renderer.domElement); //생성된 캔버스를 실제 화면에 추가

4. suno AI 이용 '오롯이케이크송' 노래 생성

메인 페이지에 있는 3D 공간의 홍보성을 극대화시키고, 브랜드의 정체성을 더욱 강하게 전달하기 위해, 누구나 쉽게 따라 부를 수 있는 귀엽고 발랄한 동요 스타일의 오롯이 케이크 브랜드의 테마송을 제작했다.

테마송의 주제인 설렘과 행복이 가득한 생일 파티에 맞게 chatGPT에게 작사를 부탁했다.

[Verse 1]

오늘은 네가 주인공~
반짝반짝 촛불이 켜졌네
오롯이 케이크 위에~
달콤한 꿈이 올라갔죠 🍰

[Chorus]

생일 축하해~ 너를 위해~
세상에서 제일 예쁜 케이크~
스푼 한 입에 웃음이 뿜!
우리 함께 노래해요~ ♪

[Verse 2]

딸기도 사르르~ 초코도 툭툭~
하얀 생크림 구름 같아요 ☁️
하나 둘 셋~ 소원을 빌어
우리 맘도 케이크처럼 달콤해~

[Chorus]

생일 축하해~ 너의 날에~
오롯이 만든 특별한 케이크~
반짝반짝 별처럼 빛나
오늘 너는 최고예요~ ★

[Ending]

오~ 오~ 로~ 롯~ 시~ ♪

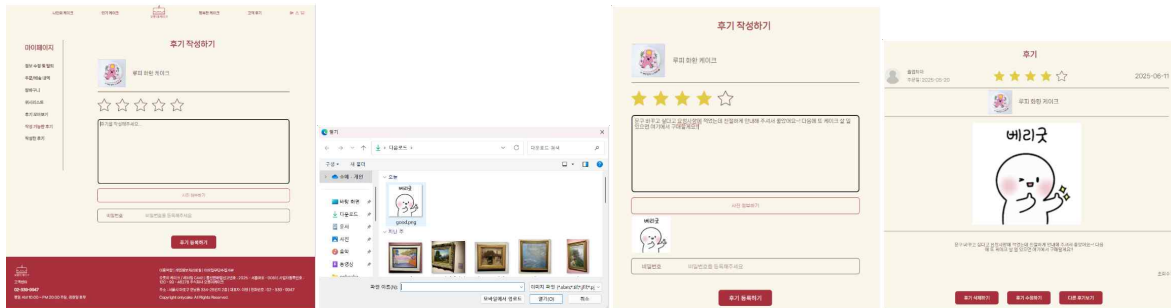
너를 위한~ 케이크~ 🎵
함께라서~ 더 행복한~
너의 생일~ 축하해요~ 🎂

chatGPT가 생성한 가사를 바탕으로 멜로디를 생성하기 위해 Suno AI를 활용했다.
가사의 따뜻하고 경쾌한 분위기를 명확히 전달하기 위해, 아래와 같은 키워드를 프롬프트로 입력했다.

"Cute, heartwarming, and childlike tone, similar to a birthday-themed nursery rhyme"

이 프롬프트는 귀엽고 동화적인 멜로디를 구현해내고, 생일파티라는 테마와 어울리는 발랄한 동요 스타일의 테마송을 생성하는 데 효과적이었다.

5. 후기 게시판 이미지 업로드 기능 구현



위와 같이 사용자가 후기를 작성할 때 이미지를 업로드 할 수 있는 기능을 넣었다.

❖ 후기 작성 페이지 (InsertReview.jsp)

- HTML Form 구성

```
<form method="post" action="InsertReview_Ok.jsp" enctype="multipart/form-data"
class="review-form">    //enctype="multipart/form-data" : 이미지를 포함한 데이터를
전송할 수 있음
```

- 파일 입력 필드

```
<input type="file" id="photoUpload" name="photoUpload" accept="image/*">
//사용자가 이미지를 선택할 수 있도록 <input type="file"> 태그를 사용하였고
accept="image/*" : 이미지 파일만 선택 가능하도록 제한
```

- 이미지 미리보기 기능

```
document.getElementById("photoUpload").addEventListener("change", function(event) {
    const file = event.target.files[0];    //선택한 파일 1개를 가져옴
    const fileName = document.getElementById("fileName");
    const previewImage = document.getElementById("previewImage");

    if (file) {
        fileName.textContent = `${file.name}`;    // 파일 이름 표시
```



```

const reader = new FileReader();
reader.onload = function(e) {
    previewImage.src = e.target.result;    // base64 데이터로 미리보기 설정
    previewImage.style.display = "block";
};
reader.readAsDataURL(file);    //파일을 base64 형식의 문자열 (Data URL) 로 읽는다.
*예시: data:image/png;base64,iVBORw0KGgoAAAANSUhEUgA...
} else {
    fileName.textContent = "";
    previewImage.style.display = "none";
}
});

```

❖ 후기 작성 완료 페이지 (InsertReview_Ok.jsp)

- 업로드 경로 설정

```

String uploadPath = application.getRealPath("/onlycake/uploads");
File uploadDir = new File(uploadPath);
if (!uploadDir.exists()) {
    uploadDir.mkdirs();    // uploads 폴더가 없다면 새로 생성
}

```

- 파일 업로드 처리 준비

```

DiskFileItemFactory factory = new DiskFileItemFactory();
ServletFileUpload upload = new ServletFileUpload(factory);
upload.setSizeMax(10 * 1024 * 1024);    // 업로드 최대 크기: 10MB
List<FileItem> formItems = upload.parseRequest(request);

```

Apache Commons FileUpload 라이브러리를 사용한다.

- 파일 저장 로직

```

if (!item.getName().isEmpty()) {
    String fileName = new File(item.getName()).getName();    // 원래 파일 이름
    String ext = fileName.substring(fileName.lastIndexOf("."));    // 확장자 추출
    String base = fileName.substring(0, fileName.lastIndexOf("."));    // 이름 부분
    String newFileName = base + "_" + UUID.randomUUID().toString().substring(0, 8) + ext;
    photoPath = newFileName;    // DB에 저장할 값

    File storeFile = new File(uploadPath + File.separator + newFileName);
    item.write(storeFile);    // 서버에 실제 파일 저장
}

```

원본 파일 이름으로 중복 방지를 위해 UUID를 붙여 새 이름을 생성하고, photoPath 변수에 새 파일 이름을 담고, 나중에 DB에 저장한다.

- DB에 이미지 경로 저장

```

String sql = "INSERT INTO review (...) VALUES (?, ?, ?, ?, ?, ?, 0, ?)";
pstmt.setString(5, photoPath);    // 업로드한 이미지 파일명

```

이미지 자체는 DB에 저장하지 않고, 파일명 (또는 경로)만 저장한다. 나중에 방식으로 불러와 사용된다.

❖ 리뷰 페이지 (Review.jsp)

- 업로드 된 이미지 파일 경로 불러오기

// 후기 조회 SQL

```

String sql = "SELECT r.reviewIdx, r.prnNo, r.memNickname, r.rating, r.reviewText, " +
    "r.photoPath, r.reviewAt, c.prnName, c.prnImage " +
    "FROM review r JOIN cake c ON r.prnNo = c.prnNo WHERE 1=1";

```

- 이미지 불러오기

```

<td class="review_photo">

```

```
<a href="ReviewDetail.jsp?reviewIdx=<%= rs.getInt("reviewIdx") %>">
    <% if (photoPath != null && !photoPath.trim().isEmpty()) { %>
            //이미지 업로드
폴더에 담긴 이미지 출력
    <% } else { %>
            //photoPath 값이 null이거나 빈
값일 경우에는 기본 이미지 (no_image.png)를 대신 보여줌
    <% } %>
</a>
</td>
```

6. 메인페이지에서 3d 공간에 있는 스피커 클릭시 음악 재생 기능 구현



3D 스피커는 블렌더에서 손수 제작했고 이 3D공간을 웹 환경에서 시각화하고 제어할 수 있게 [Three.js](#)를 이용하여 구현했다.

❖ 메인 페이지 (Index.jsp)

- GLTFLoader를 사용해 스피커 모델 로드

```
loader.load('/onlycake/object/speaker3.glTF', function (glTF) {  
    const speaker = glTF.scene;           // glTF 파일에서 scene(스피커 전체 객체) 가져옴  
    scene.add(speaker);                   // 3D 씬에 추가
```

- 스피커 내부의 각 mesh(3D 구성요소)에 클릭 가능한 속성 설정

```
speaker.traverse((child) => {  
    if (child.isMesh) {  
        child.userData.clickable = true; // 나중에 클릭 이벤트에서 확인할 수 있게 표시  
    }  
});
```

- 마우스 클릭 시 이벤트 등록

```

window.addEventListener('click', (event) => {
  const mouse = new THREE.Vector2();
  mouse.x = (event.clientX / window.innerWidth) * 2 - 1;    // -1 ~ +1 범위로 변환
  mouse.y = -(event.clientY / window.innerHeight) * 2 + 1;

  const raycaster = new THREE.Raycaster();                  // 클릭 위치에서 광선 발사
  raycaster.setFromCamera(mouse, camera);                   // 클릭 방향 설정
  const intersects = raycaster.intersectObjects(speaker.children); // 스피커의 자식

```

요소와 충돌 감지

```

if (intersects.length > 0) {    // 클릭한 곳이 스피커 mesh인 경우
  if (audio.paused) {
    audio.play().catch(error => {
      console.error('Audio playback failed:', error); // 재생 실패시 로그 출력
    });
  } else {
    audio.pause();              // 이미 재생 중이면 정지
  }
}
});
});

```

- 오디오는 JavaScript의 Audio 객체로 재생함

```

let audio = new Audio('/onlycake/audio/2.mp3'); // 음악 경로
audio.loop = true;                             // 반복 설정

```

- 음악은 텍스트 안내 영역(#playButton)클릭 시 재생/정지 가능하도록 되어 있음

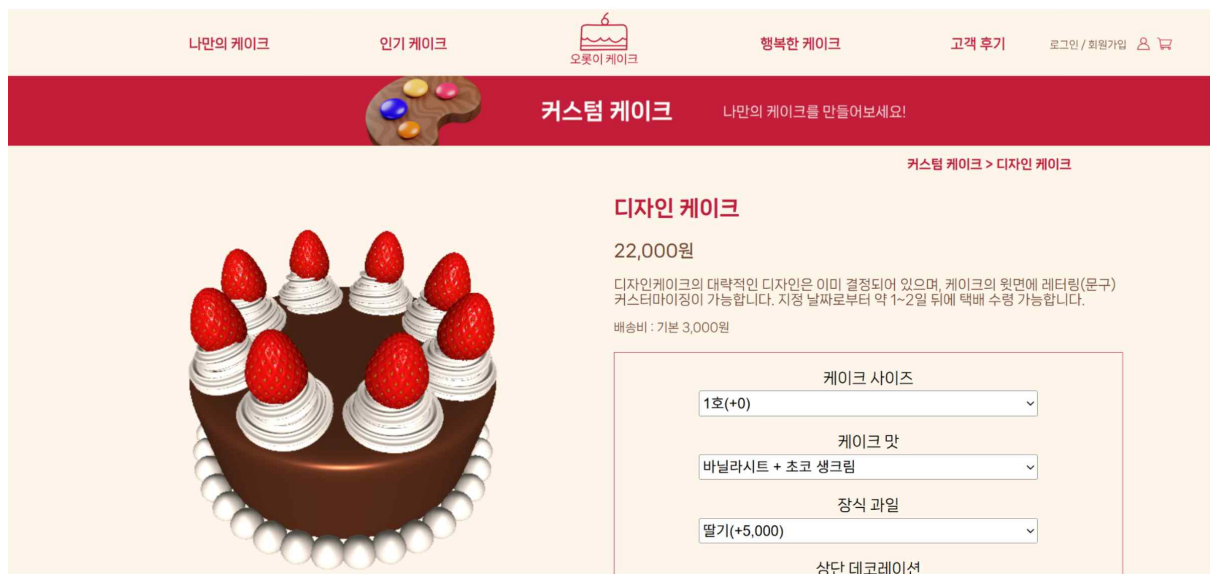
```

document.getElementById('playButton').addEventListener('click', () => {
  if (audio.paused) {
    audio.play();
  } else {
    audio.pause();
  }
});

```

```
}
});
```

7. 디자인케이크 페이지에서 옵션 선택에 따라 3d 물체 변경하는 기능



❖ 디자인 케이크 페이지 (CakeDesign.jsp)

- 공통 모델 로딩 함수

```
function loadModel(modelPath, modelType) {
    // 선택된 모델 타입에 따라 기존 3D 모델을 씬에서 제거함
    if (modelType === 'fruit' && currentFruitModel) {
        scene.remove(currentFruitModel); // 기존 과일 모델 제거
    } else if (modelType === 'topCream' && currentTopCreamModel) {
        scene.remove(currentTopCreamModel); // 기존 상단 크림 모델 제거
    } else if (modelType === 'bottomCream' && currentBottomCreamModel) {
        scene.remove(currentBottomCreamModel); // 기존 하단 크림 모델 제거
    } else if (modelType === 'cakeBase' && currentCakeBaseModel) {
```

```
scene.remove(currentCakeBaseModel); // 기존 케이크 시트 모델 제거
}
```

- GLTFLoader로 새로운 3D 모델 로딩

```
loader.load(
  modelPath, // 불러올 glTF 모델 경로
  function (gltf) {
    // 모델 타입에 따라 해당 위치에 새 모델 저장하고 씬에 추가
    if (modelType === 'fruit') {
      currentFruitModel = gltf.scene;
      scene.add(currentFruitModel);
    } else if (modelType === 'topCream') {
      currentTopCreamModel = gltf.scene;
      scene.add(currentTopCreamModel);
    } else if (modelType === 'bottomCream') {
      currentBottomCreamModel = gltf.scene;
      scene.add(currentBottomCreamModel);
    } else if (modelType === 'cakeBase') {
      currentCakeBaseModel = gltf.scene;
      scene.add(currentCakeBaseModel);
    }
  },
  undefined, // 로딩 진행률 콜백 (사용하지 않음)
  function (error) {
    console.error('모델 로드 중 오류 발생:', error); // 실패 시 오류 출력
  }
);
}
```

- 옵션 선택 시 모델 교체 이벤트 예시 (과일 선택 기준)

```
document.getElementById("cake_fruit").addEventListener("change", function() {
  const selectedValue = this.value; // 사용자가 선택한 과일 옵션
```

```
let modelFile = ''; // 해당 옵션에 대응하는 glTF 파일 경로 변수
```

```
switch (selectedValue) {  
  case '딸기(+5,000)':  
    modelFile = '/onlycake/object/Strawberry.glTF'; break;  
  case '체리(+5,000)':  
    modelFile = '/onlycake/object/Cherry.glTF'; break;  
  case '망고(+5,000)':  
    modelFile = '/onlycake/object/Mango.glTF'; break;  
  case '과일 추가 안함':  
    // "과일 추가 안함" 선택 시 기존 과일 모델만 제거하고 함수 종료  
    if (currentFruitModel) {  
      scene.remove(currentFruitModel);  
      currentFruitModel = null;  
    }  
    return; // 함수 종료  
  default:  
    return;  
}
```

```
// 선택된 모델 경로를 공통 로딩 함수로 전달
```

```
loadModel(modelFile, 'fruit');
```

```
});
```

```
// 다른 선택 항목(상단/하단 크림, 케이크 맛 등)도 동일한 구조로 되어 있음
```

- 처음 페이지 진입 시 기본 모델 로드

```
// 페이지 로딩 직후 기본으로 보여줄 3D 모델 3종 로드
```

```
loadModel('/onlycake/object/Cakebreadwhite.glTF', 'cakeBase'); // 기본 케이크 시트
```

```
loadModel('/onlycake/object/Topcream1.glTF', 'topCream'); // 상단 크림
```

```
loadModel('/onlycake/object/Bottomcream1.glTF', 'bottomCream'); // 하단 크림
```

```
//아무것도 선택하지 않은 초기 상태에서 보여줄 기본 구성
```


//추후 옵션을 바꾸면 각 요소는 scene.remove() → scene.add() 로 교체됨

- 애니메이션 및 카메라 설정

```
function animate() {  
    requestAnimationFrame(animate); // 매 프레임마다 이 함수 다시 호출 (루프 실행)  
  
    controls.update(); // OrbitControls의 변화(회전, 확대 등) 적용  
    renderer.render(scene, camera); // 씬을 카메라 시점에서 그리기  
}  
animate(); // 애니메이션 루프 시작  
//실시간으로 3D 모델이 회전, 확대되도록 렌더링 반복 실행  
//사용자 마우스 조작(OrbitControls) 결과가 반영됨
```

- 구조 요약

요소	설명
<select>	사용자가 옵션을 고름 (맛, 과일, 크림 등)
addEventListener("change")	값이 바뀌면 JS 이벤트 발생
loadModel(path, type)	해당 옵션의 3D 모델을 불러오는 함수
scene.remove()	기존 옵션 모델을 씬에서 제거
scene.add()	새로 선택한 모델을 씬에 추가

8. 택배 발송 지정 날짜 제한하는 캘린더 기능 구현

상품 발송 날짜 선택

2025-월-일

2025년 06월 ▾

↑ ↓

일	월	화	수	목	금	토
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	1	2	3	4	5
6	7	8	9	10	11	12

지우기

오늘

사항

면 최대한 맞춰서 제작해드

잇부분 글씨)

다. 비워두시면 공백으로

0자 이내

❖ 디자인 케이크 페이지 (CakeDesign.jsp), 도안케이크
페이지(CakeDrawing.jsp), 상세페이지(detail.jsp)

```
<input type="date" id="date" name="prdDeliverydate" required>
```

위의 input type="date" 필드에 JavaScript로 min, max 속성을 동적으로 설정함.

```
window.addEventListener("DOMContentLoaded", function() {  
    const dateInput = document.getElementById("date"); // 날짜 input 요소 가져오기
```

```
if (!dateInput) {  
    console.error("[오류] 날짜 입력 필드를 찾을 수 없습니다.");  
    return;  
}
```

```
const today = new Date(); // 현재 날짜 객체
```

- 날짜 제한 범위 설정

```
// 최소 날짜: 오늘로부터 5일 후
```

```
const minDate = new Date(today.getTime() + (5 * 24 * 60 * 60 * 1000));
```

```
// 최대 날짜: 오늘로부터 28일 후
```

```
const maxDate = new Date(today.getTime() + (28 * 24 * 60 * 60 * 1000));
```

```
// 날짜를 YYYY-MM-DD 형식으로 바꾸는 함수
```

```
const formatDateString = (date) => {  
    const year = String(date.getFullYear());  
    const month = String(date.getMonth() + 1).padStart(2, '0');  
    const day = String(date.getDate()).padStart(2, '0');  
    return `${year}-${month}-${day}`;  
};
```

```
// 변환된 문자열 날짜
```

```
const formattedMinDate = formatDateString(minDate); // ex "2025-06-18"
```

```
const formattedMaxDate = formatDateString(maxDate); // ex "2025-07-11"
```

```
// 날짜 input에 최소/최대 선택 범위 지정
```

```
dateInput.setAttribute("min", formattedMinDate);
```

```
dateInput.setAttribute("max", formattedMaxDate);
```

```
});
```

9. 금액 3자리수마다 콤마 처리하는 기능 구현



❖ 기념일 케이크 페이지 (CakeDesign.jsp), 도안케이크

- JSP 서버 사이드 코드에서 `java.text.NumberFormat` 클래스를 사용해서 처리

<%

`NumberFormat formatter = NumberFormat.getInstance(Locale.KOREA);` // 숫자 포맷 객체 생성 (한국 형식: 천 단위마다 쉼표)

`int prdPrice = rs.getInt("prdPrice");` // DB에서 정수형 가격 가져옴 (예: 15000)

`String formattedPrice = formatter.format(prdPrice);` // 15,000 형식으로 문자열 변환
%>

- 전체 동작 흐름 (단계별)

단계	내용
1단계	사용자가 보는 상품은 MySQL DB에 저장된 정수형 가격(<code>prdPrice</code>)을 가지고 있음
2단계	JSP 서버단에서 <code>rs.getInt("prdPrice")</code> 로 가격을 가져옴
3단계	<code>NumberFormat.getInstance(Locale.KOREA)</code> 로 한국 숫자 포맷 설정
4단계	<code>formatter.format()</code> 으로 가격을 문자열로 변환 (쉼표 포함)
5단계	<code><%= ... %></code> 를 통해 HTML에 출력 (15,000원처럼 보여짐)

10. 관리자페이지 월 별 매출 통계 그래프 구현



❖ 관리자 페이지 - 통계/후기 - 매출 페이지 (ManagerStats.jsp)

JSONArray salesData = new JSONArray(); // 매출 데이터를 담은 JSON 배열

- URL 파라미터로 전달된 조회 연도 (없으면 2025로 기본 설정)

```
String selectedYear = request.getParameter("year");
if (selectedYear == null || selectedYear.trim().isEmpty()) {
    selectedYear = "2025";
}
```

- 해당 연도의 월별 매출 총합 조회 쿼리

```
String sql = "SELECT DATE_FORMAT(ordDate, '%m') AS month, SUM(ordPay) AS  
total_sales "
```

```
    + "FROM orderinfo "
```

```
    + "WHERE DATE_FORMAT(ordDate, '%Y') = ? "
```

```
    + "GROUP BY month ORDER BY month";
```

```
pstmt = conn.prepareStatement(sql);
```

```
pstmt.setString(1, selectedYear); // ? 에 selectedYear 바인딩
```

```
rs = pstmt.executeQuery();
```

```
int[] monthlySales = new int[12]; // 1~12월 매출 저장용 배열 (기본값 0)
```

```
// 결과 집계
```

```
while (rs.next()) {
```

```
    int monthIndex = Integer.parseInt(rs.getString("month")) - 1; // 0부터 시작하는  
배열 인덱스로 맞춤
```

```
    monthlySales[monthIndex] = rs.getInt("total_sales"); // 해당 월 매출 삽입
```

```
    System.out.println("[확인] " + selectedYear + "년 " + (monthIndex + 1) + "월  
매출: " + rs.getInt("total_sales"));
```

```
}
```

- JSON 배열로 변환 (Chart.js에서 사용)

```
for (int i = 0; i < 12; i++) {
```

```
    JSONObject sales = new JSONObject(); // 월별 매출을 담은 JSON 객체
```

```
    sales.put("month", (i + 1) + "월"); // "1월", "2월", ...
```

```
    sales.put("total_sales", monthlySales[i]); // 매출 합계
```

```
    salesData.add(sales); // 전체 데이터 배열에 추가
```

```
}
```

```
System.out.println("[확인] 최종 매출 데이터 (" + selectedYear + "년): " +  
salesData.toJSONString());
```

```

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try { if (rs != null) rs.close(); } catch (SQLException e) {}
        try { if (pstmt != null) pstmt.close(); } catch (SQLException e) {}
        try { if (conn != null) conn.close(); } catch (SQLException e) {}
    }
}
%>

```

<script>

- **JSP에서 전달받은 월별 매출 JSON 데이터를 JavaScript 변수로 저장**

```
const salesData = <%= salesData.toJSONString() %>;
```

- **월 라벨과 데이터 배열로 분리**

```

const labels = [];
const data = [];
salesData.forEach(item => {
    labels.push(item.month);           // 예: "1월", "2월"
    data.push(item.total_sales);      // 각 월 매출 숫자
});

```

```
console.log("라벨:", labels); // 콘솔 디버깅
```

```
console.log("데이터:", data);
```

- **만약 모든 월의 매출이 0이라면 "판매 실적 없음" 메시지 표시**

```

if (data.every(value => value === 0)) {
    document.getElementById("monthlySalesChart").style.display = "none";
    document.getElementById("no-data-message").style.display = "block";
} else {

```

- **Chart.js 라이브러리로 막대 그래프 생성**

```
const ctx = document.getElementById("monthlySalesChart").getContext("2d");
```



```

new Chart(ctx, {
  type: "bar", // 막대 그래프
  data: {
    labels: labels, // x축 라벨: "1월", "2월", ...
    datasets: [{
      data: data, // y축 값: 매출액
      backgroundColor: "#744731",
      borderColor: "#744731",
      borderWidth: 1
    }]
  },
  options: {
    responsive: true,
    plugins: {
      legend: { display: false }, // 범례 없음
      tooltip: {
        callbacks: {
          label: function(tooltipItem) {
            return tooltipItem.raw.toLocaleString() + " 원"; // 툴팁: 숫자에
      콤마 추가
          }
        }
      }
    },
    scales: {
      y: {
        beginAtZero: true,
        ticks: {
          stepSize: 50000,
          callback: function(value) {
            return value.toLocaleString() + " 원"; // y축 라벨 포맷
          }
        }
      }
    }
  }
});

```

```
    }  
  }  
}  
}  
});  
}  
</script>
```